

PATRONES DE DISEÑO

OBJETIVOS

- ① Conocer cuáles son los patrones de diseño estructurales.
- ① Identificar cuándo aplicar cada uno de los patrones de diseño estructurales.

Dennis S. Cohn Muroy, Mag. Ing.
Pontificia Universidad Católica del Perú



1

PATRONES DE DISEÑO ESTRUCTURALES

Detallemos algunos de los patrones ...

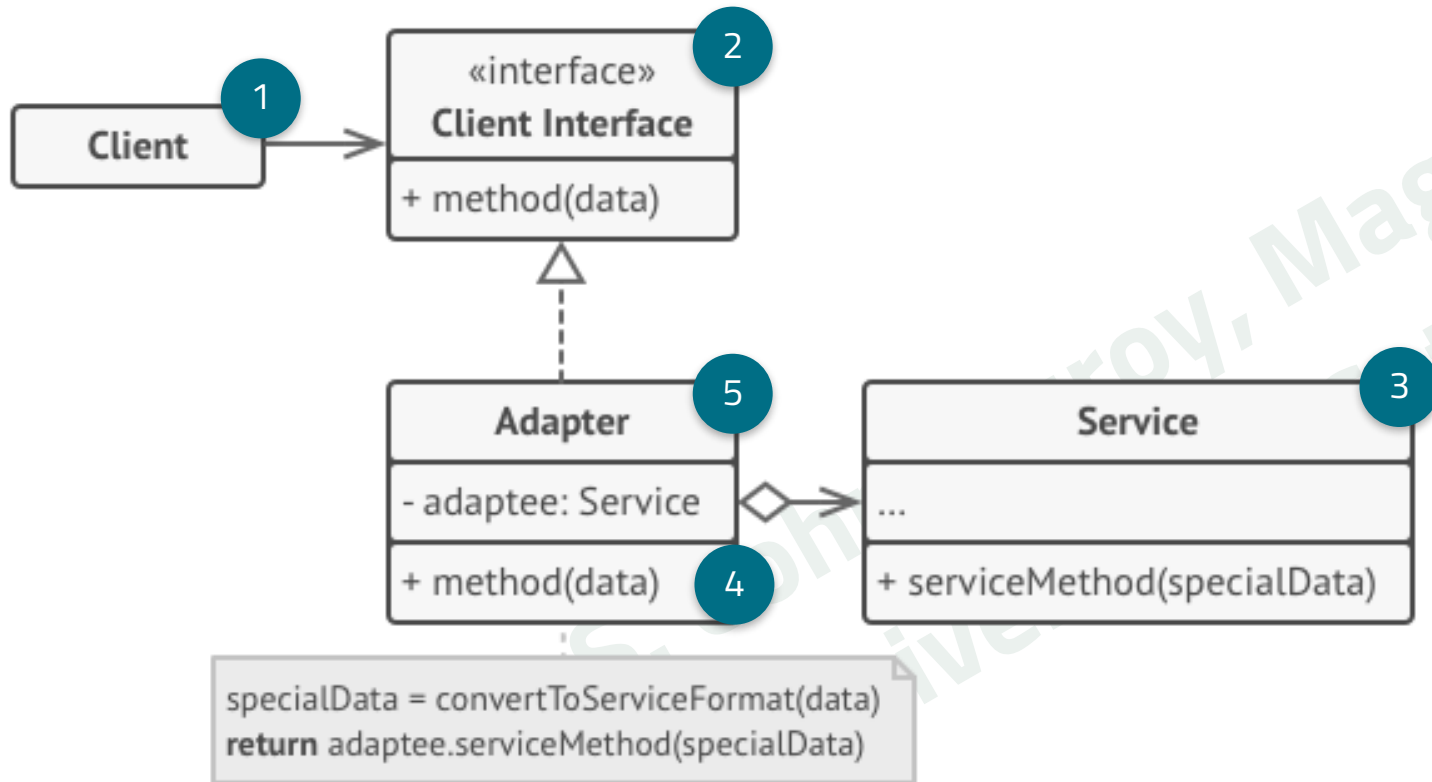
PATRÓN: ADAPTER

PROBLEMA QUE CUBRE

- Utilizar una clase que no es compatible con el resto del código.

¿CÓMO LO HACE?

Permite que dos objetos con interfaces incompatibles puedan comunicarse a través de un "Adaptador".



- 1 Cliente tiene cierta lógica de negocio pre-existente.
- 2 La interfaz describe los protocolos que otras clases deben de seguir.
- 3 Clase (posiblemente externa) que no es compatible con la interfaz.
- 4 Adaptador entre el Cliente (implementa su interfaz) y el Servicio (al cual referencia).
- 5 La clase Adaptadora puede ser reemplazada de forma transparente al Cliente.

CONSECUENCIAS: ADAPTER

- ⦿ (+) Separa la interfaz o el código de conversión de datos de la lógica de negocio del programa.
- ⦿ (+) Puedes introducir nuevos tipos de adaptadores al programa de forma transparente para el Cliente.
- ⦿ (-) Mayor complejidad debido a un grupo de nuevas interfaces y clases.

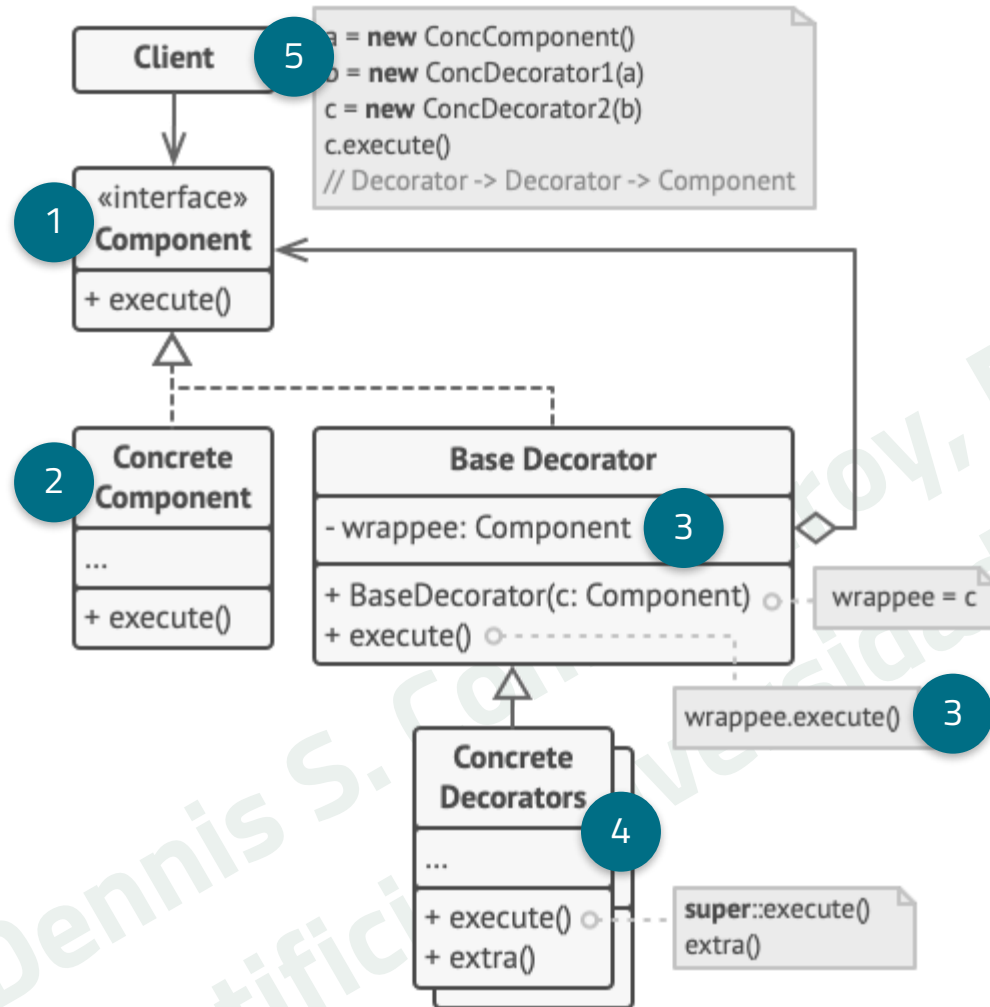
PATRÓN: DECORATOR

PROBLEMA QUE CUBRE

- ➊ Agregar funcionalidades a algunos objetos, no a toda la clase.
- ➋ Remover funcionalidades a algunos objetos.

¿CÓMO LO HACE?

Permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores (wrappers) que contienen estas funcionalidades.



- 1 Interfaz común para objetos encapsulados y que encapsularán.
- 2 Define el comportamiento básico que los decoradores extenderán.
- 3 Encapsula al objeto concreto. Delega operaciones al objeto concreto.
- 4 Definen funcionalidades adicionales y/o sobrescriben funciones del decorador base.
- 5 El Cliente crea al objeto y lo asocia a los decoradores.

CONSECUENCIAS: DECORATOR

- ⦿ (+) Permite extender el comportamiento de un objeto sin crear una nueva subclase.
- ⦿ (+) Permite añadir o eliminar responsabilidades de un objeto durante el tiempo de ejecución.
- ⦿ (+) Permite dividir una clase monolítica que implementa muchas variantes posibles de comportamiento, en varias clases más pequeñas
- ⦿ (-) Complejo remover un decorador de una pila de decoradores.
- ⦿ (-) Comportamiento del decorador depende de su orden en la pila.

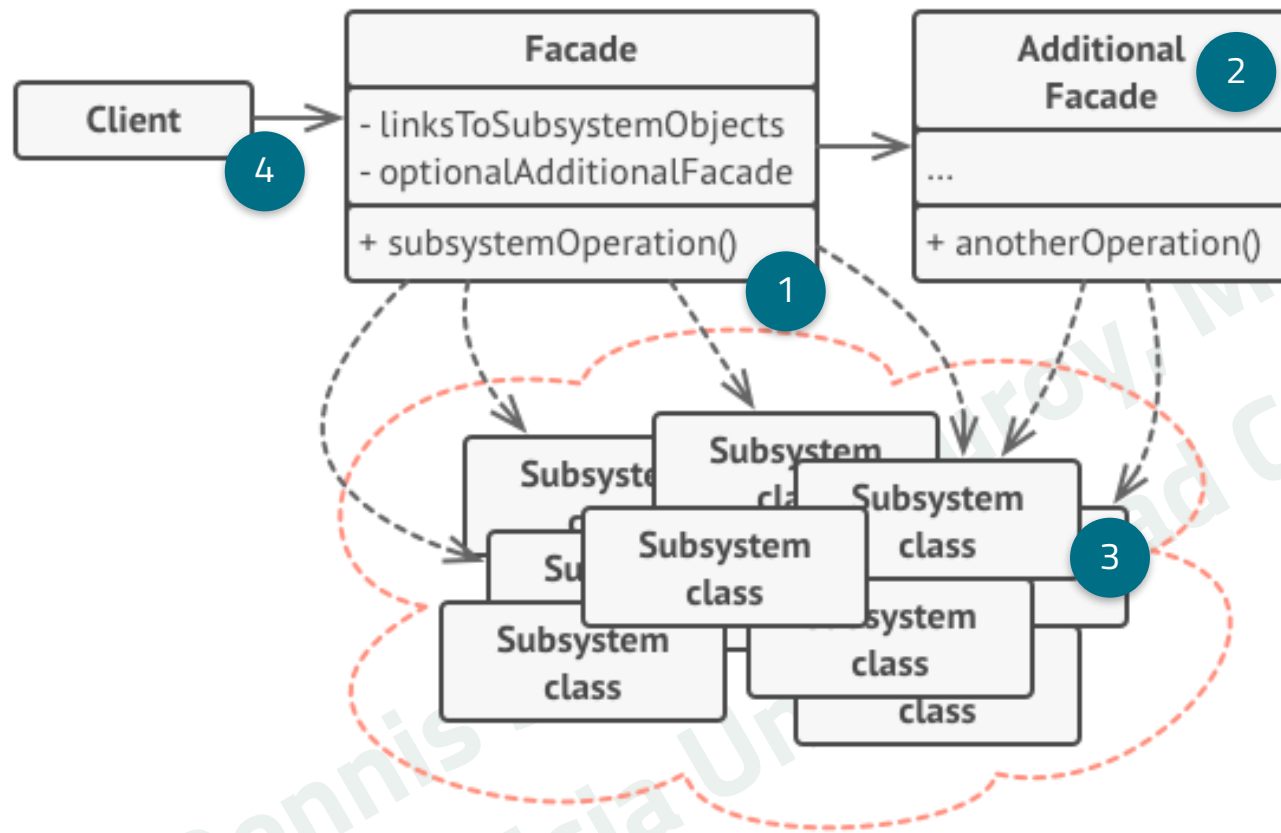
PATRÓN: FACADE

PROBLEMA QUE CUBRE

- ⦿ Contar con una interfaz para un subsistema complejo.
- ⦿ Estructurar un subsistema en capas.

¿CÓMO LO HACE?

Provee una interfaz simplificada a un conjunto de librerías, a un framework y/o a un conjunto complejo de clases.



- 1 Provee acceso a una parte particular del subsistema.
- 2 Elemento opcional para separar la complejidad de la lógica de la Fachada.
- 3 Objeto del subsistema que se comunican entre ellos; pero desconocen la existencia de una Fachada.
- 4 Cliente utiliza la Fachada en lugar de invocar directamente a los objetos del subsistema.

CONSECUENCIAS: FACADE

- ⦿ (+) Permite aislar el código Cliente de la complejidad de un subsistema.
- ⦿ (-) Puede convertirse en un gran objeto que acople a todas las clases de la aplicación.

Dennis S. Cohn Muroy, Mag. Ing.
Pontificia Universidad Católica del Perú

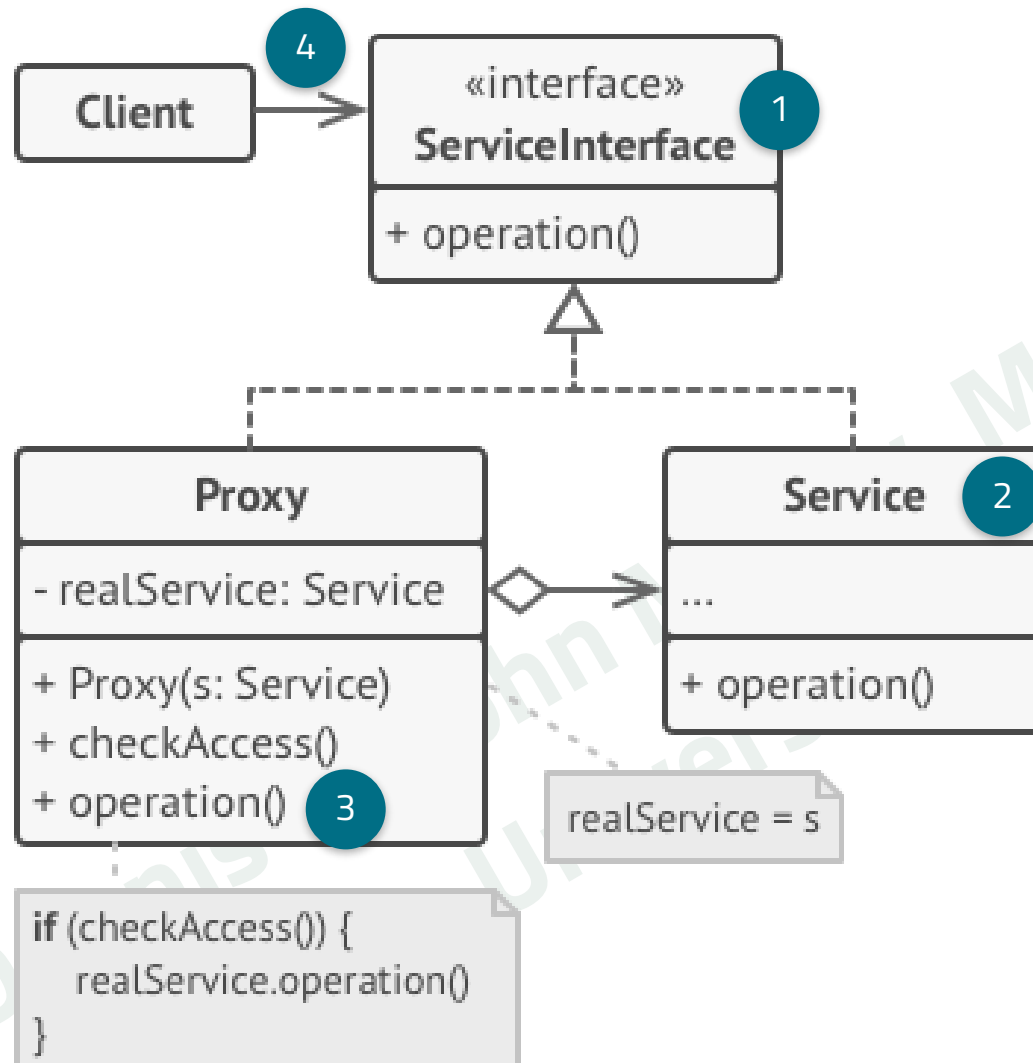
PATRÓN: PROXY

PROBLEMA QUE CUBRE

- Creación de objetos “pesados”.
- Control de acceso.
- Gestionar logs.
- Manejo de caché.

¿CÓMO LO HACE?

Controla el acceso al objeto original. Ejecuta código antes o después de invocar a los métodos del objeto original.



- 1 El proxy debe seguir esta interfaz para poder camuflarse como objeto de servicio.
- 2 Lógica de negocio a "Ocultar".
- 3 Cuando finaliza su procesamiento (inicialización diferida, registro, gestión de acceso y de caché), pasa la solicitud al servicio.
- 4 El Cliente debe de trabajar con la interfaz para poder reemplazar el Servicio con un Proxy.

CONSECUENCIAS: PROXY

- ⦿ (+) Permite controlar el objeto de servicio sin que los clientes lo sepan.
- ⦿ (+) Funciona si el objeto de servicio no está listo o no está disponible.
- ⦿ (+) Permite introducir nuevos proxies sin cambiar el servicio o los clientes.
- ⦿ (-) Mayor complejidad por el mayor número de clases.
- ⦿ (-) Puede afectar los tiempos de respuesta del servicio.

2

REFERENCIAS

BIBLIOGRAFÍA

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: elements of reusable object-oriented software. Pearson Deutschland GmbH.
- Freeman, E., & Robson, E. (2020). Head First Design Patterns. O'Reilly Media.
- Design patterns and refactoring. (n.d.). https://sourcemaking.com/design_patterns

Créditos:

- Plantilla de la presentación por [SlidesCarnival](#)
- Fotografías por [Unsplash](#)
- Diseño del fondo [Hero Patterns](#)